

Gestor de Reservas

Aplicación de consola (CLI) en Java · Práctica de fundamentos

Este taller es tu reto integrador del Módulo 1. Vas a construir, desde cero, una aplicación de consola para gestionar las **reservas de un espacio real**. Aquí pones en práctica **todo** lo que vimos: variables, tipos de datos, operadores, condicionales, ciclos, métodos con parámetros y retorno, arreglos, Scanner, validación de entradas y organización del código en clases.

La regla de oro de este taller

Aquí **no encontrarás el código resuelto**. Encontrarás algo más valioso: la especificación completa de qué debe hacer cada parte y cómo pensarla.

Tu trabajo es **escribir el código tú misma / tú mismo**. Ese es justo el músculo que evalúa el módulo.

1. El problema de negocio

“Marta Peluquería” es una peluquería. Hoy, doña Marta anota las citas en un cuaderno. El problema: a veces **agenda dos clientes a la misma hora**, pierde el hilo de cuántos cupos le quedan en el día y, al cerrar, no sabe cuánto facturó.

Antes de escribir una sola línea, responde para ti:

Piénsalo primero

Si tú tuvieras que llevar esas reservas en la cabeza durante todo un día... ¿qué datos **mínimos** necesitarías guardar de cada cita para que nada se cruce?

¿Y qué regla usarías para decidir si una hora *ya está ocupada* o está libre?

Ese cuaderno desordenado es exactamente lo que tu programa va a resolver. La computadora nunca se le olvida un cupo ni suma mal la caja.

2. Qué vas a construir

Una aplicación CLI que corre en la terminal y muestra un menú. Doña Marta escribe un número y la app responde. Debe permitir:

- **Agendar una reserva** (nombre del cliente, hora y servicio).
- **Listar todas las reservas** del día.
- **Cancelar una reserva** por su número.
- **Ver el reporte del día**: total de citas y dinero facturado.
- **Salir** del programa de forma ordenada.

Hilo conductor

Fíjate en el patrón: es la **misma estructura** que el Gestor de Inventario de la clase pasada — menú, validación, arreglos paralelos y operaciones — pero aplicada a un negocio distinto.

Si reconoces el patrón, ya tienes medio taller resuelto. Lo nuevo es **la lógica de horarios**.

3. La arquitectura (tu punto de partida)

Vas a organizar el código en **cuatro clases**, cada una con una responsabilidad clara. Todas usan **métodos estáticos** (aún no vemos objetos; eso llega en el Módulo 2).

Archivo	Responsabilidad	Qué contiene
App.java	Es el arranque. Contiene el main. Muestra el menú en un ciclo y llama a las demás clases.	El ciclo principal del programa. El Scanner compartido.
Menu.java	Todo lo que el usuario ve y escribe. Imprime opciones y lee la elección.	Método que dibuja el menú. Método que lee la opción del usuario.
Validador.java	El portero. Decide si un dato sirve o no. Nunca guarda datos; solo responde sí/no.	¿La hora está entre 8 y 18? ¿El texto está vacío? ¿Ese cupo ya está ocupado?
Operaciones.java	El cerebro. Guarda y transforma las reservas. Aquí viven los arreglos paralelos.	Agendar, listar, cancelar. Calcular el reporte del día.

Sobre los arreglos paralelos

Como aún no vemos objetos, guardarás cada dato en su **propio arreglo**, y la posición los conecta:

`clientes[0] → "Laura"`

`horas[0] → 10`

`servicios[0] → "Corte"`

La reserva de Laura vive “repartida” en el índice 0 de los tres arreglos. Ese índice es el hilo que los une.

4. Datos y reglas del negocio

Los servicios y sus precios

Para simplificar, maneja estos tres servicios con precio fijo (en pesos):

Código	Servicio	Precio
1	Corte de cabello	\$25.000
2	Tinte	\$60.000
3	Manicure	\$30.000

Las reglas que tu programa debe respetar

- El horario de atención es de **8:00 a 18:00**. Solo se agenda en horas en punto (8, 9, 10 ... hasta 17).
- Una hora **no puede tener dos reservas**. Si ya está ocupada, se rechaza.
- El nombre del cliente **no puede estar vacío**.
- El servicio debe ser 1, 2 o 3. Cualquier otro número se rechaza.
- La agenda tiene un **cupo máximo** (usa una constante, por ejemplo 10 reservas).

5. Construcción paso a paso

Vamos por bloques. En cada uno te digo **qué método crear, en qué archivo va y qué debe hacer** — pero la lógica interna la escribes tú. Compila y prueba después de cada bloque; no acumules errores.

Bloque 1 · El esqueleto que arranca

Meta: que el programa **corra, muestre el menú y no se caiga**, aunque las opciones todavía no hagan nada.

Paso 1.1 — Crea los archivos

Crea las cuatro clases vacías en tu carpeta de proyecto:

```
App.java  Menu.java  Validador.java  Operaciones.java
```

Paso 1.2 — En App.java: el ciclo del menú

Dentro del `main`, necesitas un ciclo que se repita hasta que el usuario decida salir. Piensa: ¿qué **ciclo** sirve cuando no sabes cuántas veces se repetirá?

Piénsalo

Un ciclo que corre “mientras el usuario no elija salir” encaja perfecto con un **while** (o **do-while**, que garantiza mostrar el menú al menos una vez).

- Declara **un solo** Scanner y compártelo (buena práctica: no abras varios).
- Llama a un método de `Menu` que imprima las opciones.
- Lee la opción y usa un **switch** para decidir qué hacer.

Paso 1.3 — En Menu.java: mostrar y leer

Crea dos métodos estáticos:

Método	Qué recibe / retorna	Qué hace
<code>mostrarMenu()</code>	Sin parámetros. No retorna nada (void).	Imprime las 5 opciones numeradas.
<code>leerOpcion(Scanner)</code>	Recibe el Scanner. Retorna un int.	Lee y devuelve el número que escribió el usuario.

 **Prueba de este bloque**
Compila y ejecuta. Debes ver el menú, poder escribir un número y que el programa **no se cierre** hasta que elijas “Salir”. Las demás opciones pueden imprimir “En construcción”.

Bloque 2 · El portero (Validador)

Antes de guardar cualquier reserva, hay que validar. Construye el Validador **antes** que las operaciones: así, cuando llegues a agendar, ya tienes las defensas listas.

Crea estos métodos estáticos en `Validador.java`. Todos **retornan boolean** (verdadero = el dato sirve):

Método	Recibe	Regla que verifica
<code>horaValida</code>	un int (la hora)	Que esté entre 8 y 17 inclusive. Pista: operadores relacionales + lógico (&&).
<code>nombreValido</code>	un String	Que no sea nulo ni quede vacío al quitar espacios.
<code>servicioValido</code>	un int	Que sea 1, 2 o 3.

 **Piénsalo**
Un método que verifica algo y responde “sí o no” casi siempre retorna un **boolean**. ¿Notas cómo el nombre del método (“...Valido”) ya sugiere que la respuesta es verdadero o falso?

Bloque 3 · El cerebro (Operaciones) — la memoria

Aquí viven los datos. En `Operaciones.java`, declara los arreglos paralelos y un contador como **variables estáticas de clase** (fuera de los métodos, para que todos las compartan).

- Un arreglo para los **nombres** de clientes (String).
- Un arreglo para las **horas** (int).
- Un arreglo para los **códigos de servicio** (int).
- Una **constante** con el cupo máximo (final).
- Un **contador** de cuántas reservas hay ahora mismo.

💡 Nota honesta de andamiaje

Usar variables estáticas para “recordar” datos es un **andamio temporal**. En el Módulo 2, cuando veamos objetos, esta memoria vivirá de forma más elegante dentro de cada objeto. Por ahora, es la herramienta correcta para lo que sabemos.

Bloque 4 · Agendar una reserva

Este es el corazón del taller. Crea `agendar(...)` en `Operaciones`. Piensa el flujo como una lista de decisiones **en orden**:

1. ¿Hay cupo? Si el contador llegó al máximo, avisa y no agendes.
2. Pide y valida el nombre (usa tu Validador).
3. Pide y valida la hora.
4. Pregunta: ¿esa hora ya está ocupada? Recorre el arreglo de horas con un ciclo y compara.
5. Pide y valida el servicio.
6. Si todo pasó, guarda cada dato en su arreglo, en la posición del contador, y **aumenta el contador**.

🤔 Piénsalo — el reto del cupo ocupado

Para saber si la hora 10 ya existe, necesitas **recorrer** las reservas que ya hay y comparar una por una. ¿Qué estructura recorre un arreglo? Un **for** que vaya de 0 hasta el contador.

Consejo de buenas prácticas: esa búsqueda merece su **propio método** en Validador o en Operaciones, para no repetir código. (Principio: una función, una responsabilidad.)

Bloque 5 · Listar las reservas

Crea `listar()`. Recorre los arreglos desde 0 hasta el contador e imprime cada reserva de forma legible.

- Si no hay ninguna reserva, imprime un mensaje amable (“Aún no hay reservas”) en lugar de una lista vacía.
- Muestra el número de reserva (el índice + 1, más humano), el nombre, la hora y el nombre del servicio.

🤔 Piénsalo

Tienes el código del servicio (1, 2, 3) pero quieres mostrar “Corte”, “Tinte”, “Manicure”. ¿Cómo traduces un número a un texto? Un **switch** o if-else que devuelva el nombre encaja muy bien — y como se reutiliza, ponlo en su propio método.

Bloque 6 · Cancelar una reserva

Crea `cancelar(...)`. El usuario indica el número de reserva a cancelar. Este es el bloque **más retador** — piénsalo con calma.

🔧 El reto de “tapar el hueco”

Si cancelas la reserva del índice 1 y tenías 4, te queda un **hueco** en la mitad de los arreglos.

Estrategia sencilla: desde la posición borrada, **desplaza** cada elemento un lugar hacia atrás (el de la derecha ocupa el de la izquierda) y al final **reduce el contador**.

Pista: un **for** que copie `arreglo[i] = arreglo[i+1]`. Recuerda hacerlo en los **tres** arreglos a la vez.

- Valida primero que el número exista (que esté entre 1 y el contador). Si no, avisa y no hagas nada.

Bloque 7 · El reporte del día

Crea `reporte()`. Recorre las reservas y calcula:

- El **total de citas** agendadas (¿no es simplemente tu contador?).
- El **dinero facturado**: suma el precio según el servicio de cada reserva. Usa un acumulador que arranca en 0.

🤔 Piénsalo

Un **acumulador** es una variable que empieza en 0 y va sumando dentro de un ciclo. Es el mismo patrón que usarías para sumar las notas de un curso o el total de una factura.

6. Errores que probablemente verás (y cómo leerlos)

Equivocarse es parte del oficio. Estos son los errores típicos de este taller y qué te están diciendo:

Se lee así	Qué significa	Por dónde revisar
<code>ArrayIndexOutOfBoundsException</code>	Tocaste una posición del arreglo que no existe.	¿Tu ciclo va más allá del contador? ¿Sumaste bien al agendar?
<code>InputMismatchException</code>	Esperabas un número y el usuario escribió texto.	Cómo lees con Scanner; considera validar la entrada.
<code>NullPointerException</code>	Usaste algo que aún no tenía valor.	¿Inicializaste los arreglos con un tamaño?
<code>cannot find symbol</code>	Un nombre mal escrito, o el método no existe donde lo llamas.	Revisa mayúsculas y que el método sea static y público.

💡 Si tu editor marca rojo pero el código compila

A veces VS Code subraya en rojo (“cannot find symbol”) aunque `javac` compile sin problema. Es la caché del analizador, no tu código.

La terminal manda: si `javac` compila, estás bien. Para limpiar el falso rojo: paleta de comandos → **Java: Clean Java Language Server Workspace**.

7. Retos extra (si terminas antes)

¿Ya funciona todo? Sube el nivel. Estos retos **no tienen pistas** — es tu momento de resolver solo/sola:

1. **Buscar por cliente:** una opción que reciba un nombre y muestre todas sus citas.
2. **Editar una reserva:** cambiar la hora de una cita existente (validando que la nueva hora esté libre).
3. **Horas disponibles:** mostrar qué horas del día siguen libres.
4. **Servicio más pedido:** recorrer las reservas y decir cuál servicio se agendó más veces.

8. Cómo entregar

7. Verifica que `javac *.java` compile **sin errores**.
8. Prueba cada opción del menú al menos una vez, incluyendo casos malos (hora ocupada, nombre vacío).
9. Sube tu proyecto a un repositorio de **GitHub** con un README breve que explique cómo ejecutarlo.
10. Comparte el enlace del repositorio.

Lista de verificación antes de entregar

- ☐ Las 4 clases compilan juntas.
- ☐ No se puede agendar dos veces la misma hora.
- ☐ Rechaza nombres vacíos y horas/servicios inválidos.
- ☐ Cancelar no deja huecos en los arreglos.
- ☐ El reporte suma bien el dinero.
- ☐ El código está limpio, con nombres claros y sin repetición innecesaria.

Recuerda: **no se trata de que te salga perfecto a la primera**, sino de que entiendas cada decisión que tomas. Ese es el verdadero entregable del Módulo 1. ¡A programar!